

Planora

Changes, architecture, and implementation guide

Purpose: Explain what was built, how each section works, and where to read the code.

Project: Full-stack project and task management application

Stack: Next.js App Router, React, TypeScript, Tailwind CSS, Better Auth, Drizzle ORM, PostgreSQL/Neon

Generated: September 16, 2026

This guide describes the current repository. Environment secrets are intentionally excluded.

1. What Planora Does

Planora is an authenticated workspace for managing projects, tasks, deadlines, and progress. A user signs in, sees a dashboard summary, manages projects and tasks, moves tasks through statuses, and uses a calendar to understand upcoming work.

Area	What it provides	Main route
Landing page	Product introduction, screenshots, feature explanation, workflow, demo entry point	/
Authentication	Email/password, Google OAuth, demo login	/auth
Dashboard	Counts, recent projects, recent tasks, quick action	/dashboard
Projects	Create, edit, delete, filter, progress display	/dashboard/projects
Tasks	Create, edit, delete, filter, list view, Kanban drag/drop	/dashboard/tasks

Schedule

Interactive monthly calendar and selected-day agenda

/dashboard/
schedule

2. High-Level Architecture

Browser | | Next.js App Router navigation v Layouts and pages in app/dashboard/ | |
Server Components fetch authenticated data v Server actions in lib/actions/actions.ts
| +--> getUser() -> Better Auth session | +--> Drizzle ORM queries | v PostgreSQL /
Neon Client components receive server data | +--> local UI state: filters, modals,
view mode +--> server actions: create, edit, delete, status changes +--> dnd-kit:
Kanban drag/drop

Pages own server-side data loading. Interactive components use "use client" and receive typed data as props. This keeps database access on the server while allowing responsive UI interactions in the browser.

3. Folder Map

Folder/file	Responsibility
app/page.tsx	Landing page composition.
app/layout.tsx	Root HTML, fonts, theme initialization, global body.
app/dashboard/ layout.tsx	Authenticated dashboard shell, desktop sidebar, mobile bottom navigation, independent scrolling.
app/ui/home/	Landing sections and product screenshot gallery.
app/ui/dashboard/	Greeting, summary metrics, recent projects, recent tasks.
app/ui/projects/	Project cards, progress, project modal, projects page.
app/ui/tasks/	Task page, list rows, Kanban board, task modal.
app/ui/schedule/	Interactive month calendar and loading state.
app/ui/searchbox.tsx	Command-style search across projects and tasks.

<code>lib/actions/</code>	Server actions for authentication and database mutations/queries.
<code>lib/auth.ts</code>	Better Auth configuration.
<code>db/schema.ts</code>	PostgreSQL schema definition for users, sessions, accounts, projects, and tasks.

4. Landing Page

The landing page is composed in `app/page.tsx`:

```
Nav
Hero
Benefits
Features
ProductPreview
Solutions
FinalCta
```

Navigation

`app/ui/home/nav-bar.tsx` provides desktop section links, an authentication CTA, and a mobile menu. It is a client component because the mobile menu uses React state.

Hero and demo login

`app/ui/home/hero.tsx` contains the main product message, dashboard screenshot, normal authentication link, and the live demo button. The demo button calls `signInDemo()`, which uses the configured test account and redirects to the dashboard after a successful session.

```
const result = await signInDemo();
if (!result.user) throw new Error("The demo account is not available yet.");
router.push("/dashboard");
```

Important: The demo account must exist in the database. Do not expose production credentials in source code. For a real deployment, use a dedicated seeded demo account and protect or rate-limit the demo login.

Product screenshot gallery

`app/ui/home/product-preview.tsx` uses the existing images in `public/`:

- `dash-dark.png`: dashboard overview.

- `projects-dark.png`: project management view.
- `tasks-dark.png`: task management view.
- `schedule-dark.png`: schedule/calendar view.

The gallery uses a large dashboard preview and three responsive product cards. On mobile, cards stack; on desktop, they form a three-column product tour.

Other landing sections

- `benefits.tsx`: concise value summary.
- `features.tsx`: projects, tasks, progress, and schedule capabilities.
- `solution.tsx`: the four-step flow from project creation to completion.
- `final-cta.tsx`: final conversion action and footer.

5. Authentication

Authentication is provided by Better Auth with a Drizzle adapter.

Configuration

`lib/auth.ts` sets:

- `baseUrl` from `BETTER_AUTH_URL`.
- `basePath`: `"/api/auth"`.
- Email/password authentication.
- Google social provider.
- `nextCookies()` integration.

The route handler at `app/api/auth/[...all]/route.ts` exposes Better Auth GET and POST handlers.

Email flow

```
AuthClientPage -> signIn(email, password) -> auth.api.signInEmail() -> Better Auth
creates/loads session -> router.push("/dashboard")
```

Google flow

`signInSocial()` asks Better Auth for an authorization URL and returns it to the client. The browser

navigates to that URL. This avoids treating a server redirect as a normal return value.

Google Cloud Console must include the matching callback URL, normally:

```
http://localhost:3000/api/auth/callback/google
```

Session reuse

`getUser()` is wrapped with `React.cache()`. This memoizes the authenticated user within a request so the dashboard layout and data actions do not repeat the same session lookup unnecessarily.

6. Dashboard

The dashboard page loads projects and tasks in parallel:

```
const [projectData, taskData] = await Promise.all([
  getProjectData(),
  getTaskData(),
]);
```

This avoids waiting for the project query before beginning the task query.

Dashboard layout

`app/dashboard/layout.tsx` protects the workspace by checking `getUser()`. On large screens, the layout has a fixed viewport-height sidebar and independently scrollable main area. On small screens, the desktop sidebar is hidden and `MobileNav` provides a fixed bottom navigation bar.

Summary cards

`dashboard-overview.tsx` computes:

- Total projects.
- Total tasks.
- Tasks in progress.
- Tasks in review.
- Completed tasks.

The cards are responsive: one column on small screens, two columns at medium widths, and five columns on large screens.

Recent panels

`recent-projects.tsx` and `recent-tasks.tsx` display a maximum of five items each. They retain “View All” links so the dashboard stays compact without hiding data.

7. Projects

The projects page loads project rows and compact progress aggregates in parallel. A project card contains:

- Project name and type.
- Priority badge.
- Completed/total task progress.
- Description.
- Edit and delete actions.

`getProjectProgress()` uses a grouped database query instead of loading every task row into the projects page. This reduces data transferred for a projects-only view.

Project create/edit flow

New Project button -> `ProjectView` sets `projectOpen` -> `NewProject` opens in create mode -> `createProject()` -> `getProjectData()` -> local project list refreshes Edit button -> `ProjectView` stores selected project -> `NewProject` opens with prefilled state -> `updateProject()` -> related `task.project_name` values are synchronized -> projects and tasks paths revalidate

Duplicate project names are checked client-side before mutation, excluding the project currently being edited.

8. Tasks and List View

`tasks-overview.tsx` owns the task state and shared handlers. It supports filters:

- All Tasks
- To Do
- In Progress
- In Review

- Done

The list view uses a deliberately wide table inside an overflow container. This keeps all task fields available on mobile without forcing the whole page to overflow.

Each row supports:

- Status dropdown.
- Priority display.
- Deadline.
- Edit action.
- Delete action.

Status changes optimistically update local state, then call `editTaskStatus()` to persist the change.

9. Kanban Board and Drag/Drop

`app/ui/tasks/kanban-board.tsx` USES `@dnd-kit/core`.

Board structure

There are four droppable columns:

Column	Meaning
To Do	Ready to start
In Progress	Currently active
In Review	Needs a decision
Done	Completed work

Drag flow

PointerSensor detects a drag -> useDraggable identifies task id -> useDroppable identifies target status column -> DragOverlay renders the floating card -> onDragEnd reads over.id -> changeStatus(task.id, nextStatus) -> TaskView updates local state -> editTaskStatus() persists to database

The pointer activation distance is six pixels, preventing accidental drags when clicking a card. The drag handle is separate from edit/delete controls.

10. Schedule Calendar

The old deadline lists were replaced with an interactive month calendar in `schedule-overview.tsx`.

Calendar behavior

- Six-week grid always provides stable height.
- Previous and next month controls update the visible month.
- Today returns to the current month and selects today.
- Project deadlines and task deadlines become calendar events.
- Desktop cells show event labels.
- Mobile cells show compact event dots.
- Selecting a day updates the selected-day agenda below.

Date model

Events are normalized into a shared shape with id, title, date, type, priority, status, and project name. `dateKey()` compares local calendar dates rather than timestamps, avoiding time-of-day mismatches.

11. Search

`app/ui/searchbox.tsx` is a command-style workspace search.

- Loads project and task data in parallel.
- Searches project names.
- Searches task names, project names, and descriptions.
- Groups results into Projects and Tasks.
- Shows contextual metadata such as type, priority, status, and project.
- Supports Enter to open the first result and Escape to close.
- Uses loading skeletons and a no-results state.

Search navigates to the relevant section rather than attempting to duplicate project/task detail pages.

12. Theme Toggle

`app/components/theme-toggle.tsx` toggles the explicit `dark` class on the root document element.

- Uses a moon icon in light mode and sun icon in dark mode.
- Persists the preference as `planora-theme` in `localStorage`.
- Uses `useSyncExternalStore` to avoid state-in-effect lint problems.
- Root initialization reads saved preference or system preference.
- `globals.css` defines Tailwind's explicit dark variant.

The theme initialization script is placed inside `body`, not directly inside `html`, because a script cannot be a child of the HTML element in React's document tree.

13. Responsive Design

Surface	Small screens	Large screens
Dashboard navigation	Fixed bottom navigation with four destinations.	Fixed-width sidebar with account area.
Dashboard cards	Stacked grid.	Five-column metric layout and two-panel activity layout.
Projects	One-column cards and horizontally scrollable filters.	Two/three-column card grid.
Tasks list	Contained horizontal scroll for dense fields.	Full-width task table.
Kanban	Stacked columns.	Four-column board.
Calendar	Horizontally scrollable 7-column grid with event dots.	Full labels and selected-day agenda.
Modals	Single-column fields, viewport-bounded scroll.	Two/three-column field groups.

The dashboard shell uses independent scrolling: the sidebar is constrained to viewport height, while the main content scrolls separately. This prevents long task pages from pushing the account panel out of view.

14. Loading and Performance

- Dashboard, projects, tasks, and schedule have route-level `loading.tsx` files.
- Loading screens mirror the actual destination geometry instead of using one generic skeleton.
- Independent database requests use `Promise.all()`.
- Auth session lookup is request-memoized with `React cache()`.
- Projects use aggregate progress data rather than loading all tasks.
- Production is checked with `npm run build`; development mode is slower because Next compiles routes on demand.

15. Main Data and Mutation API

Function	Purpose
<code>getUser()</code>	Request-cached authenticated user.
<code>getProjectData()</code>	Current user's projects.
<code>getProjectProgress()</code>	Grouped total/completed task counts by project.
<code>createProject()</code>	Insert project for authenticated user.
<code>updateProject()</code>	Update project and synchronize related task project names.
<code>deleteProject()</code>	Delete user-owned project.
<code>getTaskData()</code>	Current user's tasks.
<code>createTask()</code>	Insert task with project relation and copied project name.
<code>editTask()</code>	Update task fields and project relation.
<code>editTaskStatus()</code>	Persist list/Kanban status changes.
<code>deleteTask()</code>	Delete user-owned task.

Mutations call `getUser()` and reject unauthenticated requests. Database filters include the current user id for updates and deletes.

16. Running the Project

Install

```
npm install
```

Environment

Configure the required variables in a local `.env` file. Do not commit secrets.

```
BETTER_AUTH_SECRET=...  
BETTER_AUTH_URL=http://localhost:3000  
DATABASE_URL=...  
GOOGLE_CLIENT_ID=...  
GOOGLE_CLIENT_SECRET=...
```

Commands

Command	Use
<code>npm run dev</code>	Start development server.
<code>npm run lint</code>	Run ESLint.
<code>npx tsc --noEmit</code>	Run TypeScript without emitting files.
<code>npm run build</code>	Create the production build.
<code>npm run start</code>	Start the compiled production server.

Useful verification sequence

```
npm run lint  
npx tsc --noEmit  
npm run build
```

17. How to Read the Code

1. Start at a route page such as `app/dashboard/tasks/page.tsx`.
2. Read its layout to understand shared navigation and header composition.

3. Read the page data calls and identify which server actions provide the props.
4. Open the client component that owns local state, for example `tasks-overview.tsx`.
5. Follow callbacks into `lib/actions/actions.ts` to see how mutations reach the database.
6. Check the matching loading file to understand the intended loading geometry.
7. Use `db/schema.ts` and `lib/types/types.ts` to understand data shapes.

18. Known Design and Engineering Notes

- Task data stores both `project_id` and `project_name`. Project rename logic synchronizes the copied name on related tasks.
- The schedule calendar derives events from task and project deadlines; it does not create a separate calendar table.
- The Kanban board changes task status rather than persisting card order within a status column.
- The search modal loads the complete current user's project/task lists when opened. For very large accounts, a server-side search endpoint would scale better.
- Current demo login depends on the test account existing in the database.
- Production builds have been verified during development, but browser-level interaction tests would provide additional confidence for OAuth and drag/drop.

19. Change Summary

The work completed across the project focused on turning a basic task app into a complete responsive workspace:

- Improved landing page with product screenshots and full section flow.
- Added responsive dashboard shell and mobile bottom navigation.
- Added light/dark mode persistence.
- Added project and task create/edit flows.
- Added request-cached authentication and parallel route data loading.
- Added responsive loading screens.
- Added responsive project and task redesigns.
- Replaced schedule lists with an interactive calendar.
- Added list/Kanban task views with dnd-kit drag/drop.
- Redesigned search as a command-style workspace search.

- Fixed invalid HTML theme script placement and related hydration error.
- Fixed lint and module-resolution issues found during verification.

End of guide. Source files are linked by path throughout this document so the explanation can be followed directly in the repository.